



Eindhoven University of Technology
Department of Mathematics and Computer Science
Software Engineering and Technology Research Group

Configurable Monitoring Infrastructure for ROS 2 Applications

Master's Thesis

Shikai Jin
s.jin1@student.tue.nl

Supervisor:
Dr. Ivan Kurtev

Eindhoven, June 2026

Abstract

Runtime monitoring can reveal failures and requirement violations that appear only after a robot is deployed. ROS 2 exposes typed behaviour through topics, services, and actions, but monitoring solutions differ in their observation mechanisms, data formats, property checkers, and deployment assumptions. Developers therefore repeat routine work for collection, conversion, routing, transport, and evidence whenever they build a new monitoring setup.

This thesis presents a configurable monitoring infrastructure for ROS 2 applications. A domain analysis and feature model identify the commonalities and variabilities of monitoring solutions. From this model, a reference architecture separates ROS 2 observation, common data representation, transformation, routing, checker adaptation, property checking, and result export. The prototype collects topic messages, service request and response events, and action feedback and status into a common JSON-compatible record. Converter and verdict-service plug-ins isolate property-specific logic. A deployment specification is converted into matching runtime YAML files for integrated or split checking through file or MQTT transport.

Five experiments progress from a controlled topic property to unmodified Nav2, two-robot correlation, and a deployment across Ubuntu, Raspberry Pi, and macOS. In the reported runs, verdicts matched controlled expectations, recorded inputs, mission outcomes, or Gazebo positions. Identical local and remote verdict streams show that serialization and MQTT preserved result meaning in the evaluated configurations. The generator and reuse analysis show that observation, serialization, routing, transport, and evidence handling do not have to be rewritten for each property. A high-level architectural mapping also shows that the reference architecture can accommodate the principal capabilities of four reviewed monitoring approaches, without claiming that all of their components are implemented.

The results support the proposed architecture and implementation within a focused scope: passive application-level monitoring, selected ROS 2 interfaces, up to two simulated robots, and one local network. The infrastructure provides a reusable foundation on which additional observation mechanisms, checking formalisms, transports, and operational safeguards can be added.

Contents

Abstract	1
Contents	2
1 Introduction	5
1.1 Context and Motivation	5
1.2 Problem Statement	5
1.3 Objective and Research Questions	6
1.4 Scope	6
1.5 Approach and Contributions	6
1.6 Thesis Structure	7
2 Background	8
2.1 ROS 2 Communication	8
2.1.1 Topics	8
2.1.2 Services	8
2.1.3 Actions	9
2.2 Runtime Monitoring Concepts	9
2.2.1 Online and Offline Monitoring	10
2.2.2 Passive and Active Monitoring	10
2.2.3 Centralised and Distributed Checking	10
2.2.4 Property Specification Formalisms	10
2.3 Ways to Observe ROS 2	10
2.4 Common Monitoring Tasks and Data Forms	11
2.5 Feature Models	11
2.6 Deployment Terminology	12
2.7 Transport	12
2.8 Chapter Summary	12
3 Related Work	13
3.1 Review Criteria	13
3.2 ROSMonitoring and ROSMonitoring 2.0	13
3.3 ROMoSu	14
3.4 Requirements-to-Monitor Generation for ROS 2	14
3.5 RTAMT	14
3.6 ros2.tracing	14
3.7 Network and Security Monitoring	15
3.8 Containerized and Distributed Verification	15
3.9 Digital-Twin Runtime Verification	15
3.10 Field-Based Testing	15
3.11 Focused Summary	15

3.12	Research Gaps	16
4	Towards a Reference Architecture for Configurable ROS 2 Monitoring	18
4.1	From Domain Analysis to a Feature Model	18
4.1.1	Feature Groups	18
4.2	How the Model Is Used	19
4.3	Selected Requirements	19
4.4	Reference Architecture	20
4.5	Common Data Record	21
4.6	Observation Sequence	22
4.7	Representative Deployment Views	22
4.8	Chapter Summary	23
5	Implementation	24
5.1	Implementation Overview	24
5.2	Code Organization	24
5.3	Anatomy of a Monitoring Solution	25
5.4	Runtime Data Flow	25
5.4.1	Collectors	25
5.4.2	Common Records and Serialization	26
5.4.3	Transformers and Dispatchers	26
5.4.4	Checking Graph	26
5.5	Plug-ins and DSL Adaptation	27
5.6	Deployment Specification and Configuration Generation	27
5.7	Operational Support	28
5.8	Chapter Summary	28
6	Evaluation	29
6.1	Evaluation Questions and Criteria	29
6.2	Experiment Design	29
6.3	User Workflow and Experiment Artifacts	30
6.4	E1: Topic Monitoring in an Integrated Deployment	31
6.5	E2: Topics, Services, and Actions	31
6.6	E3: One Nav2 Robot	32
6.7	E4: Two Nav2 Robots	33
6.8	E5: Three Physical Machines	33
6.9	Resource Use and Latency	34
6.10	Automation and Reused Work	34
6.11	Accommodation of Existing Approaches	35
6.12	Answer to RQ4	36
7	Discussion	37
7.1	RQ1: Capabilities and Observation Gaps	37
7.2	RQ2: Commonalities and Variabilities	37
7.3	RQ3: Reference Architecture	38
7.4	RQ4: Evaluation of the Selected Path	38
7.5	Design Trade-offs	39
7.6	Limitations	39
7.6.1	Observation and Checking	39
7.6.2	Ordering, Time, and Delivery	39
7.6.3	Configuration and Operation	40

7.7	Threats to Validity	40
7.7.1	Internal Validity	40
7.7.2	External Validity	40
7.8	Future Work	40
8	Conclusions	42
8.1	Answers to the Research Questions	42
8.2	Main Contribution	43
8.3	Availability	43
	Bibliography	44
	Appendix	45
A	Schemas and Reproducibility	46
A.1	Common Data Record	46
A.2	Deployment Specification Example	47
A.3	Plug-in Contracts	48
A.4	Repository	48

Chapter 1

Introduction

1.1 | Context and Motivation

Robots operate in changing environments. Their behaviour depends on software, hardware, sensors, communication, and the physical world. They also increasingly use artificial-intelligence components for perception and decision making. Uncertain sensor input, model confidence, and conditions that differ from training data can therefore influence a robot after deployment. Testing remains essential, but it cannot cover every environment and interaction that a robot may encounter.

Runtime monitoring complements testing by observing an execution while it happens or by checking its recorded trace afterwards. A monitor may report, for example, that commanded velocity exceeded a limit, a service call did not have the expected effect, or an action missed a deadline. Unlike model checking, runtime monitoring does not explore every possible execution path; it analyses behaviour that is observed in actual executions [1].

The Robot Operating System 2 (ROS 2) offers several useful observation points. Nodes exchange typed messages through topics, services, and actions, and lower layers expose network and execution traces [2]. The challenge is to turn these observation mechanisms into a reusable and configurable monitoring infrastructure. The next section states this problem more precisely.

1.2 | Problem Statement

A monitoring solution must make several connected choices. It must decide what to observe, how to reduce and represent the observations, where to check a property, how to transport data between processes, and where to store or present the result. Those choices change between applications and deployments. A local developer test may keep collection and checking in one process; a fielded robot may keep ROS 2-facing collection near the robot and send records to a separate verifier.

Existing approaches address important parts of this problem, but they use different observation levels, data formats, property languages, and deployment assumptions. A ROS 2 developer who builds each solution directly must repeatedly write routine code for subscriptions, service or action observation, message conversion, filtering, routing, transport, and result recording. This boilerplate obscures the application-specific decisions and makes monitoring setups harder to reproduce.

Property specification and property checking are themselves important research problems. Temporal logics, state machines, grammars, and rule languages provide different solutions; the runtime-verification literature surveys this design space [1, 3]. This thesis does not propose a new property language or checking algorithm. Instead, it focuses on the infrastructure that supplies observations to a selected checker. A checker-specific converter and verdict service are

treated as extension points.

The problem addressed by this thesis is therefore:

How can heterogeneous ROS 2 observations be collected, represented, processed, routed, and delivered to property checkers through a reusable infrastructure whose deployment and checker-specific parts remain configurable?

1.3 | Objective and Research Questions

The objective is to design, implement, and evaluate a configurable monitoring infrastructure for ROS 2 applications. The infrastructure should cover typed topic messages, service events, and action feedback and status; preserve enough context to relate a verdict to its observations; and support both integrated and split placement of collection and checking.

The objective is refined into four research questions:

- RQ1.** What monitoring capabilities and ROS 2 observation mechanisms exist, and which application-level observation gaps remain in the reviewed approaches?
- RQ2.** What are the commonalities and variabilities of monitoring solutions for ROS 2 applications?
- RQ3.** How can a reference architecture incorporate the selected monitoring features while separating reusable infrastructure from property-specific checking?
- RQ4.** To what extent do the proposed reference architecture and its implementation support the monitoring needs of representative ROS 2 applications?

1.4 | Scope

The prototype follows a deliberately focused path through the monitoring design space. It observes the application-level data that ROS 2 exposes through topics, service introspection, and hidden action feedback and status topics. It supports passive online monitoring and recorded-data replay. Checking may run inside the ROS 2-facing monitor or in a separate runtime reached through a file or MQTT transport.

The following concerns are outside the implemented scope: a new property formalism, automatic synthesis of checking algorithms, active intervention in the system under monitoring, global ordering across hosts, fault-tolerant distributed verification, network-packet inspection, and operating-system tracing. The reference architecture keeps explicit extension points for some of these concerns, but they are not claimed as prototype capabilities.

1.5 | Approach and Contributions

The research follows four steps. First, related work and ROS 2 observation mechanisms are reviewed using consistent criteria. Second, their commonalities and variabilities are organized in a feature model. Third, a reference architecture is derived and implemented as a Python prototype. Fourth, five experiments evaluate the selected feature configuration, progressing from a controlled topic property to a two-robot Nav2 deployment across three computers.

The contributions are:

1. a domain analysis and readable feature model for ROS 2 monitoring solutions;

2. a reference architecture that separates observation, common data representation, processing, routing, checker adaptation, checking, and result use;
3. a prototype that collects topic messages, service request and response events, and action feedback and status into one common record format;
4. plug-in interfaces for checker-specific converters and verdict services, with integrated and split deployment;
5. a deployment-specification generator that creates matching runtime YAML files and reduces low-level, routine monitoring boilerplate; and
6. an evaluation using controlled stimuli and Nav2 case studies, including evidence that the architecture can accommodate the main capabilities of four reviewed monitoring approaches.

1.6 | Thesis Structure

Chapter 2 introduces ROS 2 communication, monitoring concepts, observation mechanisms, feature models, and deployment terminology. Chapter 3 reviews the selected approaches and derives the research gaps. Chapter 4 presents the feature model and reference architecture. Chapter 5 explains the prototype and configuration workflow. Chapter 6 evaluates the implementation and the architectural accommodation of related work. Chapter 7 discusses the research questions, trade-offs, limitations, threats to validity, and future work. Chapter 8 gives concise final answers and conclusions.

Chapter 2

Background

2.1 | ROS 2 Communication

ROS 2 structures an application as nodes that exchange typed data. The three main application interfaces are topics, services, and actions [4, 2].

2.1.1 Topics

A topic is a named publish–subscribe channel. Publishers send messages without addressing individual receivers, and any number of subscribers may receive them. Topics are suited to streams such as sensor measurements, robot pose, or velocity commands. Delivery behaviour depends on compatible Quality of Service (QoS) policies such as reliability, durability, history, and depth [5, 6].

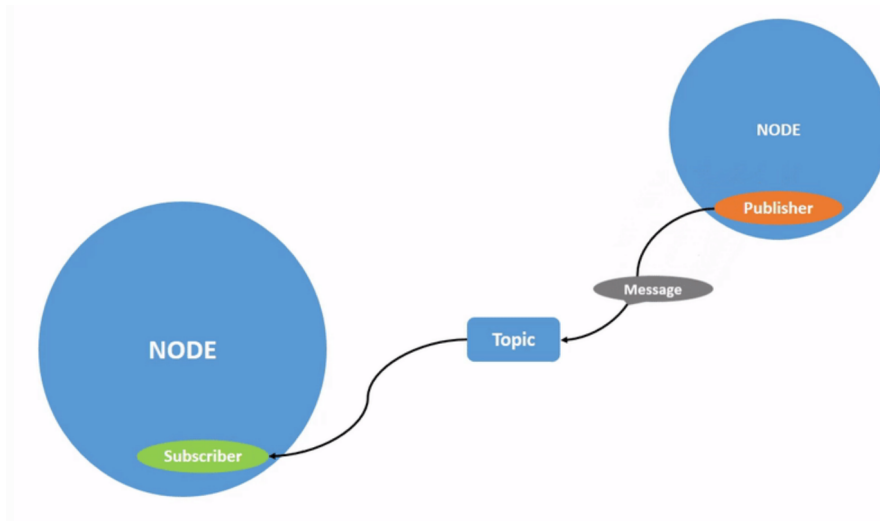


Figure 2.1: Topic communication. Author’s illustration based on the ROS 2 topic model [5].

2.1.2 Services

A service represents a request–response interaction. A client sends a typed request and a server returns a typed response. ROS 2 service introspection can publish request and response events on a hidden `/<service>/_service_event` topic when introspection is enabled by the client or server [7]. This event stream lets a monitor observe a service without becoming the client or server.

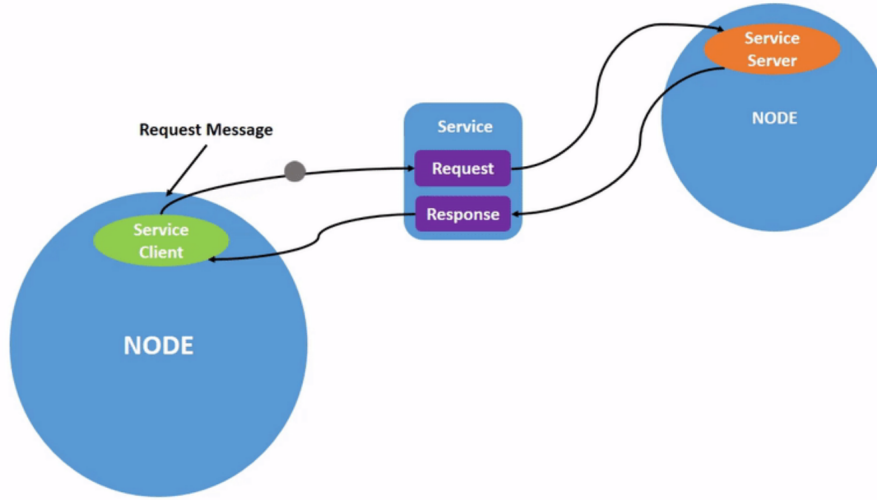


Figure 2.2: Service communication. Author's illustration based on the ROS 2 service model [4].

2.1.3 Actions

An action represents a long-running goal. It combines service interactions for goal management with hidden topics for feedback and status. A client can send a goal, receive progress feedback, request cancellation, and obtain a final result [8]. Monitoring feedback and status reveals progress and terminal goal states. A complete action trace additionally requires the hidden goal, cancellation, and result service events.

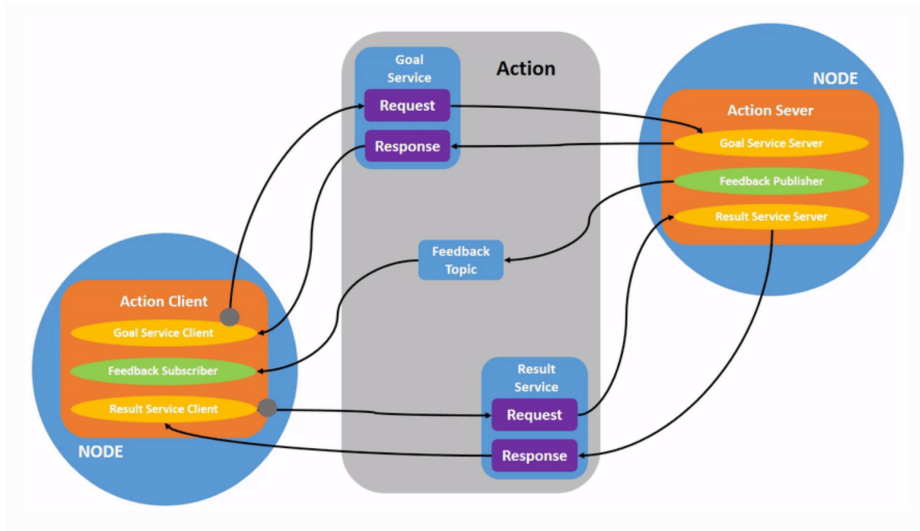


Figure 2.3: Action communication. Author's illustration based on the ROS 2 action design [8].

2.2 | Runtime Monitoring Concepts

This thesis uses the following terms consistently:

System under monitoring. The running ROS 2 application whose behaviour is observed.

Observation. A value or event obtained from the system under monitoring, such as a topic message, service response, or action status.

Property. A condition expected of the observed behaviour. A property can be expressed as a temporal-logic formula, a rule, a state machine, or another formalism.

Monitor. The component that obtains and prepares observations.

Oracle or checker. The component that evaluates a property using prepared observations.

Verdict. A checking result, normally indicating satisfaction, violation, or an inconclusive state.

2.2.1 Online and Offline Monitoring

Online monitoring processes observations during an execution. It is suitable for timely warnings and supervision. Offline monitoring checks recorded observations later and is useful for replay, debugging, and audit. The same monitoring infrastructure may support both by routing a common record either to a live checker or to persistent storage.

2.2.2 Passive and Active Monitoring

A passive monitor observes, checks, and reports without changing the behaviour of the system under monitoring. An active monitor can additionally block, replace, or trigger behaviour. Passive and active behaviour are not mutually exclusive at architecture level: the same solution may record all observations and intervene only after selected verdicts. An oracle may be used in either mode. The prototype in this thesis implements only passive monitoring.

2.2.3 Centralised and Distributed Checking

Collection and checking may run in one process, on one host in separate processes, or on different hosts. Distributed runtime verification also studies property decomposition, communication delay, clock differences, failure, and coordination [3]. The split deployment in this thesis separates collection from checking, but checking remains logically centralised in a verifier.

2.2.4 Property Specification Formalisms

Properties can be written in several forms. Temporal logics express ordering, deadlines, and time-bounded relationships. Rules express conditions and event-condition-action behaviour. Grammars describe valid event sequences, while finite-state machines describe allowed states and transitions. Stream languages compute derived values and verdicts from timestamped input. Each checker consequently expects its own input representation. This thesis remains neutral about the formalism and adapts common observations through converter and verdict-service plug-ins [1].

2.3 | Ways to Observe ROS 2

The observation mechanism determines what a monitor can see and how intrusive it is. The main categories used in this thesis are:

Native ROS 2 interfaces. Normal subscriptions, service-introspection event topics, and hidden action topics retain ROS names and types. They are well suited to application-level properties but depend on DDS discovery and compatible QoS.

Application instrumentation. Monitoring calls or wrappers are added to application code. This can expose domain events that ROS communication does not reveal, at the cost of changing the application.

Network observation. Packet capture or network probes observe traffic without normal ROS subscriptions. They can support security analysis but may have to reconstruct ROS meaning from middleware or packet data.

Tracing. ROS 2, middleware, or operating-system tracepoints observe callbacks, scheduling, middleware operations, and execution timing. This provides lower-level evidence than typed application messages.

The prototype selects native ROS 2 interfaces. The other categories appear in Chapter 3 so the selected scope is not mistaken for the whole monitoring domain.

2.4 | Common Monitoring Tasks and Data Forms

The monitoring-domain analysis by Rabiser et al. identifies recurring tasks such as definition, instrumentation, collection, processing, routing, checking, and persistency [9]. For this thesis they are organized as six operational tasks:

1. collect observations;
2. convert raw observations to a common representation;
3. transform or reduce the data;
4. route or transport it;
5. adapt it to a checker and evaluate properties; and
6. export the verdict or retain evidence.

The data changes form along this path. A *raw observation* is the ROS 2-specific message or event. A *common data record* stores its payload with source, phase, time, session, and identity information. A *checker-specific event*, called a DSL record in the prototype, contains the shape expected by one checking formalism. Monitoring frameworks often use different event formats, so the boundary between these three forms must be explicit.

2.5 | Feature Models

Chapter 4 uses a feature diagram to describe the monitoring design space; this section introduces the notation needed to read it. A feature model represents a family of related solutions as a hierarchy of features. A filled circle indicates a mandatory child for the selected configuration; an empty circle indicates an optional child. Alternative or “one-or-more” groups state how several children may be selected. Cross-tree notes record dependencies that are awkward to express in the hierarchy. Feature modelling originated in Feature-Oriented Domain Analysis and is widely used to capture commonality and variability [10, 11].

A feature model describes valid design choices; it does not by itself provide deployment details. Host addresses, ROS names, interface types, class paths, file locations, and transport parameters belong in a separate deployment specification.

2.6 | Deployment Terminology

To avoid opaque symbols, the thesis uses readable deployment labels:

Host. A physical or virtual machine on which one or more executable runtimes run.

Runtime. One executable monitor or verifier process built from a YAML configuration.

Integrated deployment. Collection, processing, checking, and result export run in one ROS 2-facing runtime.

Split deployment. A ROS 2-facing monitor runtime collects records and a separate verifier runtime checks them. They may run on one host or on different hosts.

These two deployment forms are representative endpoints, not an exhaustive catalogue. The reference architecture also permits several monitor or verifier runtimes and different placements, provided that their links are configured.

2.7 | Transport

ROS 2 communication is normally carried by DDS implementations, which provide discovery and typed data distribution [2]. Monitoring records can also cross process or host boundaries through files, sockets, web protocols, or message brokers. The reference architecture treats this carrier as replaceable.

The prototype uses MQTT in its split online deployment because MQTT is a small, standard publish–subscribe protocol with broker-based decoupling and selectable delivery QoS [12]. It lets a verifier receive JSON records without a ROS 2 installation. MQTT is therefore an implementation choice for the prototype, not a mandatory dependency of the reference architecture.

2.8 | Chapter Summary

ROS 2 exposes typed topic, service, and action observations, while instrumentation, network capture, and tracing provide other views. Monitoring solutions share collection, representation, processing, routing, checking, and evidence tasks, but vary in their property formalism and deployment. The next chapter reviews existing approaches using these dimensions.

Chapter 3

Related Work

3.1 | Review Criteria

Monitoring papers use the same terms for different goals, so each selected approach is described using the following questions:

1. What is its goal and what observation level does it use?
2. Which ROS interfaces or other sources are observed?
3. How are properties represented and checked?
4. Is checking online, offline, or both?
5. Where do collection and checking run?
6. Which parts are configurable, extensible, or generated?

Table 3.1 fixes the meaning of “level” used in this chapter.

Table 3.1: Observation levels used to classify related work.

Level	Examples of visible information
Application/ROS interface	Typed topic messages, service events, action feedback and status, and domain events.
Middleware/network	DDS operations, endpoints, packets, flows, and network security features.
Execution/system	Callback execution, scheduling, tracepoints, processes, and operating-system events.

3.2 | ROSMonitoring and ROSMonitoring 2.0

ROSMonitoring is a topic-oriented runtime-verification framework for ROS 1 [13]. A YAML description selects the topics. A generator creates monitor nodes and launch/remapping changes so selected messages pass through a monitor. The monitor converts ROS messages to JSON events. In online mode it sends events over WebSocket to an external oracle; in offline mode it records a trace. An online filtering mode can block or republish messages according to the oracle result. The property formalism is therefore supplied by the external oracle rather than fixed by the ROS-facing monitor.

ROSMonitoring 2.0 extends this approach to services and to explicitly ordered topic observations [14]. Its Runtime Monitoring Language supports event-pattern properties, including past-time Metric Temporal Logic. The reported ROS 2 support is partial, and ROS 2 action monitoring is not covered.

3.3 | ROMoSu

ROMoSu is a configurable platform for runtime monitoring of ROS-based applications [15, 16]. It separates a monitoring-configuration phase from data collection. Reusable system-under-monitoring types and monitor configurations describe topic fields, sampling, validation, storage, and presentation. At runtime, a ROS connector and adaptation manager collect data, and a runtime-data broker connects validation, persistency, dashboards, and external constraint services.

The reported implementation focuses on ROS 1 topic data. Properties may be checked by connected validation or complex-event-processing services. The platform provides an administration interface and dashboard. Its main variability lies in reusable configurations, selected data fields and rates, and external services; code generation is not its central mechanism.

3.4 | Requirements-to-Monitor Generation for ROS 2

Perez et al. present a requirements-to-monitor toolchain for ROS 2 [17]. FRET, the Formal Requirements Elicitation Tool, lets a user write structured FRETISH requirements. Ogma is a generator that translates these requirements and the selected ROS 2 inputs into monitor artifacts. Copilot in this work is a stream-based runtime-verification language and framework, not an AI programming assistant. Ogma generates Copilot code, which is compiled to C99 and packaged as a ROS 2 node.

The approach observes selected topic values at the application level and checks generated temporal properties online. Its principal automation is the translation from structured requirements to executable monitor code and ROS 2 packaging. Deployment variation beyond the generated package is not the main focus reported in the paper.

3.5 | RTAMT

RTAMT is a library for online and offline robustness monitoring of Signal Temporal Logic (STL) and related specifications [18]. A property is evaluated over timestamped signals, producing a Boolean or quantitative robustness result. Adapters can supply signals from cyber-physical and robotic applications, including ROS. RTAMT is primarily a checking engine: source collection, routing, storage, and multi-host deployment are supplied by the surrounding application rather than by a complete monitoring platform.

3.6 | `ros2_tracing`

`ros2_tracing` is a low-overhead tracing framework for ROS 2 [19]. LTTng (Linux Trace Toolkit: next generation) records tracepoints, which are predefined event probes in ROS 2, the middleware layer, and user code. The resulting traces reveal callback execution, message flow, latency, and scheduling.

This is execution- and middleware-level observation rather than a typed application-property pipeline. It supports live trace collection and offline analysis. Tracepoint selection

and analysis are configurable; the reported framework does not prescribe one property formalism or verdict service.

3.7 | Network and Security Monitoring

ROS-FM performs fast monitoring at the network level using extended Berkeley Packet Filter (eBPF) and eXpress Data Path (XDP) mechanisms [20]. It observes traffic without adding a normal ROS subscriber for every topic, applies security-oriented processing, and supports distributed visualization. The trade-off is that typed application meaning has to be recovered from lower-level traffic.

ROSIDS23 and ROSPaCe are datasets for intrusion-detection research rather than general monitoring infrastructures [21, 22]. They combine network and, in the case of ROSPaCe, ROS 2 and operating-system evidence. They show that security properties may require several observation levels, but do not define a reusable property-checking deployment workflow.

3.8 | Containerized and Distributed Verification

Aldegheri et al. generate ROS-compliant monitors and package them in Docker containers [23]. The approach studies monitor placement between edge and cloud resources. Its main contribution is portable verification deployment rather than broad ROS interface collection.

Distributed runtime verification more generally studies systems in which observations and checking are spread across components and hosts [3]. It includes property decomposition, partial observations, clock and ordering assumptions, communication cost, and failures. Separating a collector from a central verifier is one deployment form, but it is not equivalent to decentralising the property itself.

3.9 | Digital-Twin Runtime Verification

Betzer et al. place runtime verification within a cloud-located digital twin for an autonomous mobile robot under uncertainty [24]. Robot state is transmitted through MQTT. The digital twin maintains and extends this state, and monitors generated from TeSSLa (Temporal Stream-based Specification Language) properties perform runtime validation. The design considers monitor placement inside the twin or in separate services and includes a path by which checking can influence actuation. Collection, digital-twin state, TeSSLa checking, and feedback are therefore distinct architectural responsibilities.

3.10 | Field-Based Testing

Caldas et al. combine runtime verification with field-based testing for ROS-based robots [25]. Their guidance covers data collection, isolation, monitoring reactions, and use of operational evidence. It emphasizes that simulation cannot reproduce every field condition and that monitoring artifacts must support a wider testing process. The work is process guidance rather than one fixed observation and checking pipeline.

3.11 | Focused Summary

Table 3.2 summarizes the reviewed approaches with the same criteria. A dash means that the publication does not make that capability a principal part of the approach; it does not imply that the capability is impossible.

Table 3.2: Focused comparison of selected monitoring approaches.

Approach	Observation	Property and checking	Execution and placement	Configuration and automation
ROSMonitoring / 2.0	Topics; 2.0 adds services and ordering	External oracle; 2.0 also provides RML/past-time MTL	Online or offline; inserted monitor nodes	YAML selects sources; generator creates monitor/launch artifacts
ROMoSu	Selected ROS 1 topic fields	Validation and external services, including CEP	Broker-connected platform services	Reusable system and monitor configurations; UI
FRET-Ogma-Copilot	Selected ROS 2 topic values	Generated stream/temporal monitors	Compiled monitor in a generated ROS 2 package	Generation from structured requirements
RTAMT	Timestamped signals supplied by an adapter	STL robustness and Boolean checking	Online or offline library	Property and parser/API configuration
ros2_tracing	ROS 2, middleware, and system tracepoints	Trace analysis; no fixed property language	Low-overhead collection and offline analysis	Tracepoint and analysis configuration
ROS-FM	Network traffic	Security rules and traffic analysis	Distributed probe and visualization setup	Probe and rule configuration
Digital-twin RV	MQTT robot state and digital-twin state	Generated TeSSLa monitors	Cloud twin/service with optional feedback	External properties and monitor-placement choices
Rabiser et al.	Domain-independent probes and resources	Requirements and resource checking	Reference architecture with setup, execution, and support	Domain model and variation points

3.12 | Research Gaps

The review shows complementary strengths rather than one missing checking language. The gaps relevant to this thesis are:

- Typed ROS 2 topic observation is common, but the reviewed approaches do not jointly cover topic messages, service request/response events, and action feedback/status through one representation.
- Observation, processing, routing, checking, and deployment are often coupled to one platform or toolchain. Replacing a checker or changing its placement can therefore require new integration code.
- Configurability is usually concentrated in one area, such as selecting topics, writing properties, or placing generated monitors. Few approaches expose source, processing, checker, transport, output, and placement choices through one coherent model.

- Automation is available for particular transformations, especially requirements-to-monitor generation, but routine observation and multi-runtime wiring still recur across solutions.
- Evidence needs a stable identity and context if an online verdict is to be traced back to heterogeneous observations or replayed later.

These gaps motivate a reference architecture for the infrastructure between ROS 2 observations and property-specific checkers. The proposed design is presented only after this neutral review, in the next chapter.

Chapter 4

Towards a Reference Architecture for Configurable ROS 2 Monitoring

4.1 | From Domain Analysis to a Feature Model

The review in Chapter 3 shows that a monitoring solution is not defined by one checker. It combines choices about observable elements, observation and checking placement, monitoring mode, operation, and implementation. A feature model is needed to keep these choices explicit and to prevent one prototype configuration from being presented as the complete domain.

The model in Figure 4.1 was derived by coding recurring capabilities from the literature against the monitoring tasks introduced in Chapter 2. Features that occur across approaches became common variation points. Features needed to answer the research questions were then selected for the prototype. The redrawn diagram deliberately shows the main decisions rather than every implementation parameter; host addresses, ROS names, QoS values, and class paths belong to the deployment specification.

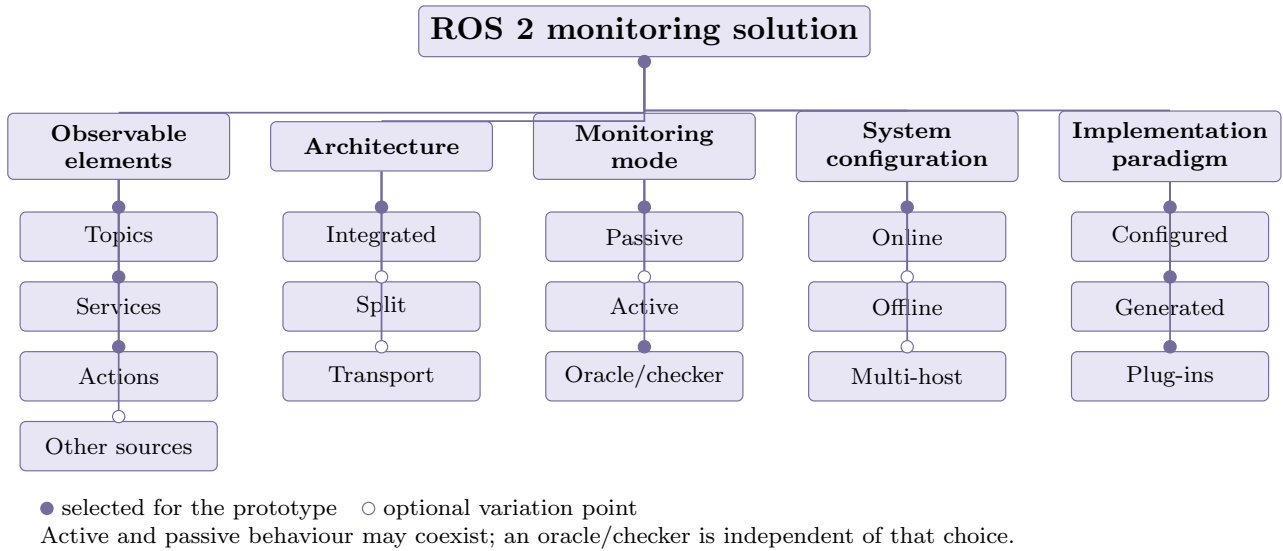


Figure 4.1: Feature model of the monitoring design space. Filled circles mark the path selected for the prototype; empty circles mark retained variation points.

4.1.1 Feature Groups

Observable elements. Application-level observation means typed data exposed by the ROS 2 graph: topics, service events, and action feedback or status. Other sources include pa-

rameters, lifecycle transitions, middleware events, network traffic, and operating-system traces.

Architecture. Collection and checking may be integrated or split, and a transport is required whenever data crosses a runtime boundary.

Monitoring mode. Passive and active behaviour may coexist. An oracle or checker is applicable in both cases: a passive solution reports its verdict, while an active solution can additionally use it for feedback.

System configuration. Monitoring may be online, offline, or both, and it may use one or several hosts.

Implementation paradigm. A solution can be configured, generated, and extended through plug-ins. These mechanisms are complementary.

Parameter changes are not observed by the current prototype. They appear only as an unselected domain option: ROS 2 publishes parameter events on a regular topic, but a dedicated configuration and evaluation would be required before claiming support.

4.2 | How the Model Is Used

Figure 4.2 separates design selection from deployment detail. A user first chooses a feature configuration. The user then provides the additional information that the feature model cannot contain: hosts, runtime roles, ROS resources and types, links, transport settings, and plug-in class paths. The generator converts this deployment specification into one runtime YAML file per executable host role.

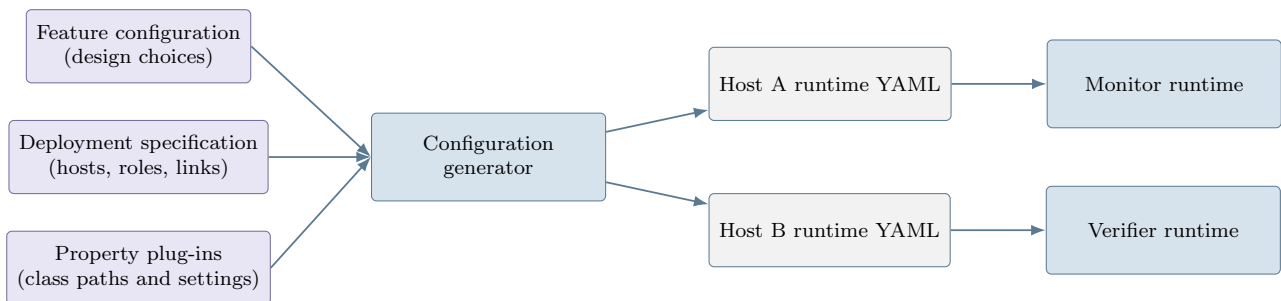


Figure 4.2: From design choices and a deployment specification to executable monitoring runtimes.

The prototype implements only the selected path: passive application-level collection, online or replayed checking, integrated or split placement, and file or MQTT transport. The reference architecture retains broader extension points, but it does not make unimplemented features automatic.

4.3 | Selected Requirements

The domain analysis and selected feature configuration yield the following requirements:

R1 — Heterogeneous observation. Topic messages, service request and response events, and action feedback and status must enter one processing model without erasing their differences.

- R2 — Common observation format.** Every collected observation must be converted to a common format that preserves source, phase, time, payload, session, and record identity.
- R3 — Configurable processing and routing.** Field selection, rate limiting, change filtering, outputs, and checking chains must be selected without changing collector code.
- R4 — Property-formalism independence.** Users must be able to plug in their own conversion to a checker event format and their own property formalism or checking service.
- R5 — Deployment variation.** The same monitoring responsibilities must support integrated and split deployment, with explicit transport configuration.
- R6 — Traceable evidence.** A verdict must be able to identify the observation records that contributed to it.
- R7 — Extensibility.** New sources, transformers, transports, converters, verdict services, and exporters must be addable behind stable interfaces.

4.4 | Reference Architecture

Figure 4.3 shows the reference architecture. The names in the diagram are the same role names used in the implementation chapter. Numbering follows one observation from left to right and then through checking.

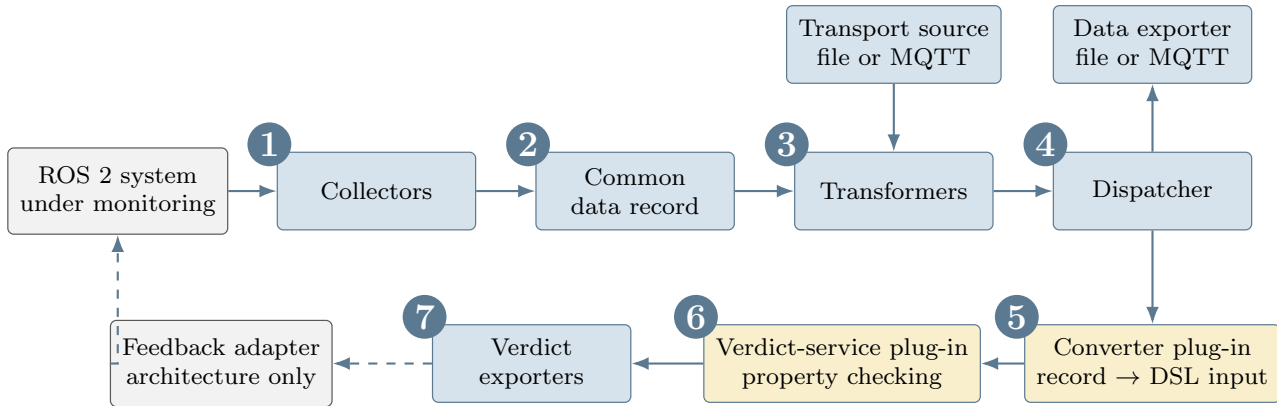


Figure 4.3: Reference architecture for configurable ROS 2 monitoring. Solid paths are implemented; the dashed feedback path is an architectural extension only.

1. A *collector* observes one typed ROS 2 source. A *transport source* performs the corresponding role for records arriving from a file or MQTT.
2. The collector creates a *common data record*. This is the first format shared by all later components.
3. Zero or more *transformers* select fields, limit rate, flatten structure, or suppress unchanged values.
4. A *dispatcher* routes a record to one or more data exporters and checking chains. Data exporters retain JSONL evidence or send records over a configured transport.

5. A *converter plug-in* changes a common data record into the input expected by a checker. A domain-specific language (DSL) is a language tailored to one problem domain, such as temporal properties or event-state patterns. The architecture calls the converter output a DSL record because every property formalism may require a different shape.
6. A *verdict-service plug-in* evaluates the property through a local implementation or an external checking engine.
7. A *verdict exporter* writes or publishes the result. The verdict retains the contributing record identifiers.

A plug-in is a user-developed Python class loaded by its module and class name. A converter implements the stable conversion interface; a verdict service implements the stable evaluation interface. To add a DSL, a developer defines the DSL-record fields, implements these two classes, tests them against sample records, and selects their class paths and settings in the deployment configuration. ROS 2 collection, transport, and evidence handling are reused.

The architecture also shows an optional feedback adapter. It could translate a verdict into a ROS 2 command, service request, or other intervention. The prototype neither implements this adapter nor claims active enforcement.

4.5 | Common Data Record

The common data record separates observation from checker representation. Table 4.1 defines its principal fields. The full schema and an example appear in Appendix A.1.

Table 4.1: Principal fields of the common data record.

Field	Meaning
<code>session_id</code> , <code>record_id</code>	Execution identity and stable observation identity.
<code>source_type</code> , <code>source_name</code>	Topic, service, or action and its ROS 2 name.
<code>source_node</code> , <code>msg_type</code>	Discovered producer/server and ROS 2 interface type.
<code>phase</code>	Empty for a normal topic; request/response or feed-back/status for heterogeneous interactions.
<code>timestamp</code> , <code>ros_timestamp</code>	Monitor receive time and an optional timestamp from the ROS message.
<code>data</code>	Recursively converted observation payload.
<code>metadata</code>	Sequence number, QoS information, and applied transformers.

ROS 2 messages can contain primitive values, nested messages, arrays, sequences, and time values. The collector recursively converts them to a JSON-compatible tree: Boolean, numeric, and string fields become JSON scalars; nested messages become objects; arrays and sequences become arrays; and ROS time uses `sec` and `nanosec` fields. The surrounding record stores type and source identity, so nesting is preserved without requiring a checker to import the original ROS 2 class. Arbitrary Python objects are not part of the interchange contract.

For example, a `geometry_msgs/msg/Twist` message is represented as nested `linear` and `angular` objects. A speed converter may retain only `linear.x`, while the original JSONL record can still be kept as evidence. Service-event metadata also carries the client identifier and sequence number needed to pair requests and responses.

4.6 | Observation Sequence

Figure 4.4 explains the normal sequence before deployment choices are introduced. A typed observation becomes a common record, the verifier emits a verdict, and records and verdicts can be stored as evidence. In an integrated deployment the monitor and verifier lifelines are inside one executable runtime; in a split deployment the common record crosses a configured transport.

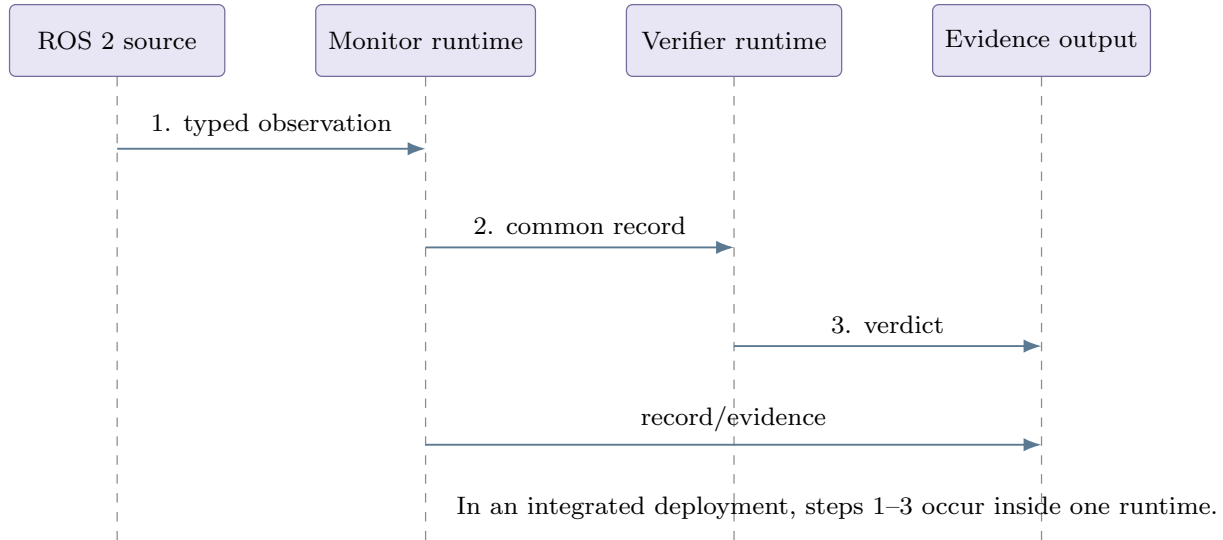
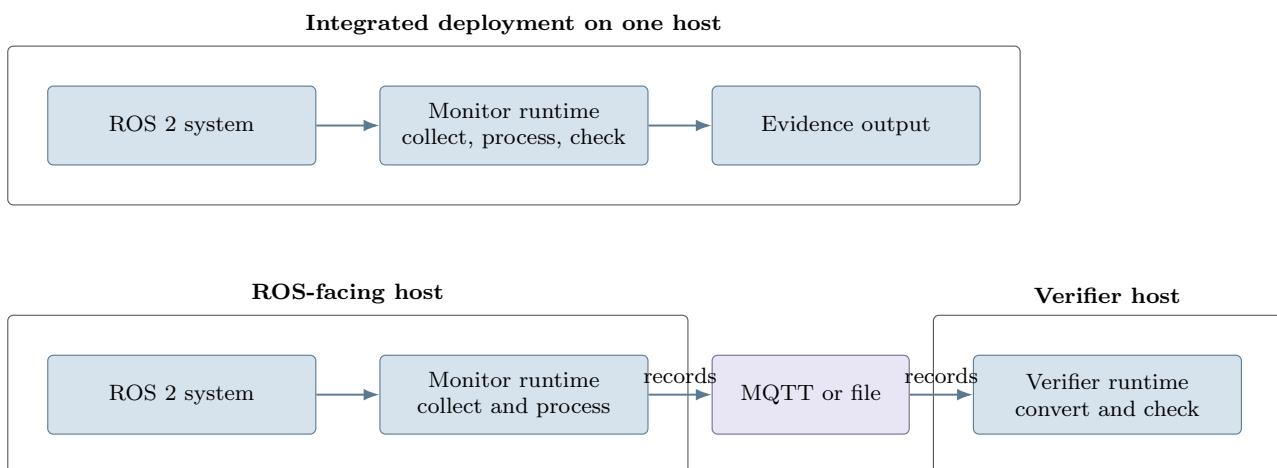


Figure 4.4: Sequence for one observation and its resulting evidence.

4.7 | Representative Deployment Views

Figure 4.5 uses host boundaries and executable runtimes rather than treating a process as a UML deployment element. A *ROS 2-facing runtime* is simply a process with ROS 2 libraries and graph access; it is not necessarily running on the robot itself.



Split deployment; the two hosts may also be the same physical machine.

Figure 4.5: Integrated and split deployments. These are representative configurations, not the only placements allowed by the architecture.

The integrated form minimizes transport boundaries. The split form isolates the checker from ROS 2 dependencies and permits a different host or operating system. More configura-

tions can be formed by adding monitor runtimes, verifier runtimes, and explicit links in the deployment specification. Such a configuration still requires network addresses, credentials, topics or file paths, and component placement; the feature model alone cannot supply them.

4.8 | Chapter Summary

The feature model distinguishes domain choices from deployment parameters. The reference architecture realizes the selected path as a sequence of collectors, common records, transformers, dispatchers, converter plug-ins, verdict-service plug-ins, and exporters. It supports integrated and split placements while keeping active feedback outside the current prototype. The next chapter explains how this design is implemented and generated.

Chapter 5

Implementation

5.1 | Implementation Overview

The prototype is implemented in Python for ROS 2 Kilted. Before individual classes are discussed, Figure 5.1 shows the complete transformation chain. A deployment specification and user plug-ins are inputs to configuration generation. The generated YAML starts either a `monitor_node` runtime, which has ROS 2 access, or a `node_runner` verifier runtime, which can run without ROS 2.

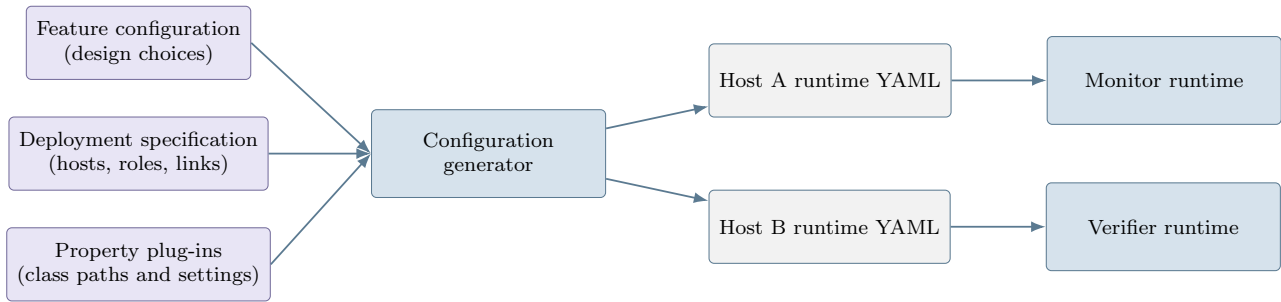


Figure 5.1: Implementation chain from authored specifications and plug-ins to generated runtime configurations and executable runtimes.

A *runtime* in this chapter means one executable process built from one runtime YAML file. It owns its configured sources, processing and checking graph, outputs, and lifecycle. A host can run more than one runtime, although the generator normally emits one consolidated configuration for each host role.

5.2 | Code Organization

The implementation is maintained in the version-controlled project at <https://github.com/Jin-Shikai/ROS2-Monitor-Infra>. The main directories are separated by responsibility:

This separation is important: code under `monitor/` is reused by every case study, while `custom/` contains the property knowledge that changes between applications.

Table 5.1: Code organization.

Path	Responsibility
monitor/	Reusable data records, collectors, transformers, dispatchers, sources, exporters, plug-in loading, runtime construction, and configuration generation.
custom/	Example application- and property-specific converter and verdict-service plug-ins.
demo/	Deterministic ROS 2 stimulus programs and demonstration material.
test/	Unit tests for reusable components and example plug-ins.
docs/	Data-record, configuration, plug-in, and deployment documentation.

5.3 | Anatomy of a Monitoring Solution

Table 5.2 distinguishes what a user authors, what the generator creates, and what is reused. A one-host integrated solution needs one generated YAML file. A split solution normally needs a monitor YAML and a verifier YAML; more hosts produce more matching configurations.

Table 5.2: Artifacts in a concrete monitoring solution.

Authored by user	Generated	Reused
One JSON deployment specification	One runtime YAML per generated host role	Collectors for topics, services, and actions
Converter and verdict-service classes when built-ins are insufficient	Consistent transport inputs/outputs and checking links	Common record conversion and JSONL serialization
Property and plug-in settings	Runtime output directories and executable commands	Transformers, dispatchers, file/MQTT transport, evidence and verdict formats

The user still decides which observations and properties matter. Automation removes the routine wiring around those decisions; it does not infer the property or deployment intent.

5.4 | Runtime Data Flow

Figure 5.2 shows the runtime graph before its components are explained. A collector or transport source supplies common data records. The same dispatcher can send a record to evidence output and to one or more checking chains.

5.4.1 Collectors

The monitor runtime builds one collector for each configured ROS 2 resource:

Topic collector. It resolves the message class at startup, creates a normal subscription, recursively converts each message to a dictionary, and creates a topic data record.

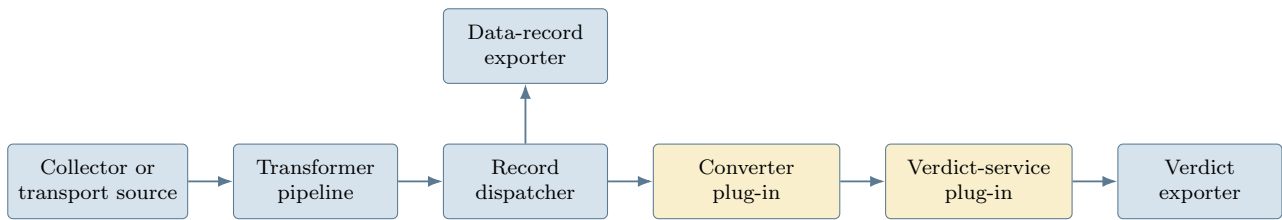


Figure 5.2: Implemented runtime data flow. Cream components are property-specific plug-ins; blue components are reusable infrastructure.

Service collector. It resolves the service event class and subscribes to the hidden service-event topic for the configured service. The client or server must enable service introspection in `CONTENTS` mode. Event type distinguishes request from response; the client identifier and sequence number are retained as metadata.

Action collector. It subscribes to the configured action’s hidden feedback and status topics. The configured phases determine which hidden topics are used. Goal, cancellation, and result interactions are services and require separate service-event collectors for a complete trace.

These mechanisms explain how the prototype “gets grip” on services and actions: it uses the event topics that ROS 2 itself exposes, not an interceptor placed between application nodes.

5.4.2 Common Records and Serialization

The `DataRecord` dataclass implements the schema from Section 4.5. It assigns a session identifier, a per-source sequence number, and a record identifier formed from the session, source type, source name, phase, and sequence. It stores monitor wall-clock time and, when present, the message header timestamp.

Serialization uses one JSON object per line. Empty optional fields are omitted. Because the payload has already been converted recursively, nested ROS messages, arrays, sequences, and primitive values remain valid JSON data. The same JSON is used for files and MQTT, so split deployment does not change the meaning of the record.

5.4.3 Transformers and Dispatchers

Each collector owns an ordered transformer pipeline. The built-in `FieldExtractor` selects nested fields, `RateThrottler` limits the maximum output frequency, and `OnChangeFilter` suppresses repeated values. A transformer can return no record to stop processing. Names of successful transformations are appended to record metadata.

A dispatcher fans one record out to any number of exporters and checking chains. Built-in data-record exporters write JSONL or publish MQTT. File and MQTT sources perform the inverse role in a verifier runtime. This separation keeps processing independent of the carrier.

5.4.4 Checking Graph

A *checking graph* is the configured directed graph from common records to converters, verdict services, and outputs. It is not a ROS graph. A converter selects one or several source records and emits a checker-specific DSL record. A verdict service evaluates that record and may emit a verdict. Links can also chain a derived record into another converter.

The following compact YAML fragment represents one graph:

```

topics:
  - name: /cmd_vel
    type: geometry_msgs/msg/Twist

converters:
  - id: speed_input
    type: custom.speed:SpeedConverter
    inputs: [/cmd_vel]

verdict_services:
  - id: speed_limit
    type: custom.threshold:ThresholdVerdict
    threshold: 0.5

links:
  - {from: "converter:speed_input", to: "verdict:speed_limit"}

```

Listing 5.1: Runtime YAML for a speed property.

The runtime builder validates component identifiers, loads the selected classes, constructs dispatchers, and starts scheduled converter work only after wiring is complete. Session-start and session-end records are retained as evidence but are not passed to property checking.

5.5 | Plug-ins and DSL Adaptation

A converter plug-in subclasses `DataConverter` and implements `convert(record)`. It may return no value, a dictionary, or a derived record. Stateful converters can correlate sources, retain windows, or schedule deadline events. A verdict-service plug-in subclasses `VerdictService` and implements `evaluate(dsl_record)`. Its result uses the common `Verdict` structure: `property_id`, Boolean result, timestamp, details, session identifiers, and contributing record identifiers.

The plug-in class is selected by an import path such as `custom.speed:SpeedConverter`. The loader imports it, verifies that it implements the required base class, and passes configured keyword arguments to its constructor. Thus “supporting a DSL” has a concrete meaning:

1. define the DSL-record fields expected by the checker;
2. implement a converter that produces those fields and retains input record identifiers;
3. implement or wrap a verdict service that evaluates the property; and
4. select both classes and their parameters in the deployment specification.

The evaluation plug-ins implement thresholds, service-effect correlation, action deadlines/status, and multi-robot separation. A temporal-logic engine could use the same interfaces, but such an integration is not claimed as part of the prototype.

5.6 | Deployment Specification and Configuration Generation

The JSON deployment specification contains:

hosts named deployment hosts;

runtimes ROS 2 sources, monitor roles, converters, and verdict-service roles placed on each host; and

links source and destination runtimes, payload type (**records** or **dsl**), and file or MQTT transport settings.

The term *deployment specification* is used because the JSON already contains placement decisions. It is not a generation “request”, and the generator directly creates YAML configuration files from it.

For each host, the generator chooses a ROS 2-facing monitor configuration or a ROS-independent verifier configuration. These start **monitor_node** and **node_runner**, respectively. Same-host links become in-process graph links. Cross-host record links create a matching data exporter and transport source; cross-host DSL links create matching output and input endpoints. Both sides derive the same MQTT topic or shared file path. The algorithm is:

Algorithm 1 Generate runtime YAML from a deployment specification

Require: deployment specification D

Ensure: runtime YAML set Y

- 1: **for all** host h in $D.hosts$ **do**
 - 2: create an empty runtime configuration y_h
 - 3: add sources and property plug-ins placed on h
 - 4: convert same-host dependencies to in-process links
 - 5: generate paired inputs and outputs for cross-host links
 - 6: validate identifiers, payload types, and supported transports
 - 7: write y_h when h contains an executable monitoring role
 - 8: **end for**
 - 9: **return** Y
-

Appendix A.2 gives a complete, small deployment specification and the generated-file interpretation.

5.7 | Operational Support

Runtime YAML can be checked before startup, and the implementation logs the resolved graph. File outputs support offline replay through **FileReplaySource**; MQTT outputs use selectable broker, port, topic, and QoS settings. The repository provides a container definition for repeatable packaging, but the heterogeneous experiment also runs a standalone verifier in a normal Python environment to demonstrate separation from ROS 2.

5.8 | Chapter Summary

The implementation turns one deployment specification and optional property-specific plug-ins into executable runtime configurations. Reusable collectors expose topics, service events, and action feedback/status as common records. Transformers, dispatchers, file/MQTT transport, and evidence handling are reused, while converters and verdict services isolate DSL and property logic. The next chapter evaluates both this data path and the saved infrastructure work.

Chapter 6

Evaluation

6.1 | Evaluation Questions and Criteria

This chapter evaluates RQ4: *To what extent do the proposed reference architecture and its implementation support the monitoring needs of representative ROS 2 applications?*

The following criteria are stated before the experiments so that the evidence to be collected is explicit:

- C1 — Observation coverage.** The prototype should collect typed topic messages, service request/response events, and action feedback/status.
- C2 — Functional correctness and evidence.** Produced verdicts should match expected state changes or independently recorded outcomes and identify contributing records.
- C3 — Deployment variation.** The same interfaces should work in integrated and split deployments, including a heterogeneous multi-host placement.
- C4 — Automation and reuse.** A deployment specification should generate matching runtime YAML files, while experiments should reuse observation, representation, processing, transport, and evidence code.
- C5 — Architectural accommodation.** The reference architecture should be able to host the principal capabilities of ROSMonitoring, ROMoSu, digital-twin runtime verification, and the Rabiser et al. reference architecture through configuration and explicit extension points.

Resource use and latency are reported as supporting observations, not as a performance benchmark.

6.2 | Experiment Design

Five experiments provide complementary rather than repetitive evidence. E1 and E2 use deterministic stimulus nodes so every transition is known in advance. E3 replaces these nodes with unmodified Nav2, a widely used ROS 2 navigation stack, in Gazebo. E4 adds two namespaces and a property using both robots. E5 keeps the E4 checking graph but distributes the application, monitor, and verifier across Ubuntu, Raspberry Pi, and macOS. The term *representative* therefore means that the set progresses from controlled interface tests to existing navigation software, multi-robot correlation, and heterogeneous placement. It does not claim to represent every robot or field condition.

Table 6.1: Overview of the evaluation experiments.

ID	Application	Deployment	Main evidence
E1	Commanded-speed stimulus	Integrated	Topic collection and local verdict generation.
E2	Speed, reset service, and Fibonacci action stimuli	Split over local MQTT	Topics, services, actions, and local/remote equivalence.
E3	One TurtleBot3 with Nav2 in Gazebo	Split over local MQTT	Unmodified navigation, routes, deadline, and action outcome.
E4	Two TurtleBot3/Nav2 instances in Gazebo	Split over local MQTT	Namespaces and a property using both robots.
E5	E4 across three computers	DDS plus MQTT on a LAN	Heterogeneous physical placement and equivalent verdict streams.

E1–E4 ran on one Ubuntu 24.04 workstation with ROS 2 Kilted. E3–E5 used Gazebo and Nav2; E1 and E2 did not simulate a robot and instead used deterministic ROS 2 stimulus processes. Split same-machine experiments used Mosquitto on port 1884. E5 used the workstation for Gazebo and Nav2, an Ubuntu Raspberry Pi for collection and the broker, and a macOS laptop for checking.

6.3 | User Workflow and Experiment Artifacts

The same five-step workflow was used in every experiment:

1. identify ROS 2 sources and properties;
2. choose integrated or split placement;
3. write one JSON deployment specification with sources, plug-in class paths, hosts, and links;
4. generate runtime YAML and start the reported commands; and
5. compare online verdicts with JSONL records, mission logs, or Gazebo positions.

Table 6.2 makes the authored/generated/reused boundary explicit. Ten runtime YAML files were generated across the five reference setups. E5 reused the E4 property logic unchanged; only placement and transport settings changed.

Table 6.2: How the framework was used in each experiment.

ID	Specified	Generated	Property-specific work	Reused infrastructure
E1	Topic, threshold, integrated placement	1 runtime YAML	Speed converter and threshold settings	Topic collector, record, graph, verdict output

ID	Specified	Generated	Property-specific work	Reused infrastructure
E2	Four sources, two properties, MQTT link	2 runtime YAML files	Reset-effect correlation; speed property reused	Service/action collectors, MQTT, JSONL, dispatch
E3	Nav2 sources, speed and navigation properties	2 runtime YAML files	Navigation deadline/status logic	Action collection, reduction, MQTT, evidence
E4	Six namespaced topics and three properties	2 runtime YAML files	Two-source separation conversion	Namespaces, topic collection, transport, verdict attribution
E5	Three hosts with the E4 graph	3 runtime YAML files	No new property logic	All E4 components; generated placement changes

6.4 | E1: Topic Monitoring in an Integrated Deployment

One workstation ran a stimulus node and the monitor as two ROS 2 processes. The collector, converter, and verdict service were integrated in the monitor. `/cmd_vel` was published at approximately 5 Hz; `linear.x` alternated between 1.85 and 0.2 m/s every three seconds. The property was

$$\text{linear.x} \leq 0.5 \text{ m/s.}$$

A verdict was expected only when satisfaction changed. During 120 seconds, 593 records contained 40 expected changes. The monitor produced all 40 verdicts, no extra verdict, and attributed each verdict to the first record on the new side of the threshold.

Table 6.3: E1 results.

Collected <code>/cmd_vel</code> records	593
Expected / produced verdicts	40 / 40
Missed or additional verdicts	0

E1 therefore establishes the complete integrated topic path for C1 and C2.

6.5 | E2: Topics, Services, and Actions

E2 ran three deterministic stimulus processes, one monitor, Mosquitto, and one verifier on the workstation. The monitor collected:

- `/cmd_vel`, with the E1 speed pattern;
- `/reset_pose` request and response events every three seconds;

- `/odom` at 5 Hz; and
- `/fibonacci` feedback and status for order-6 goals every five seconds.

Odd reset calls moved the pose near the origin; even calls deliberately reported success without moving it. The reset-effect property required distance to the origin to become at most 0.05 m within one second of a successful response. This explains why a service response and a later topic record appear in one verdict.

The speed property ran once inside the monitor and once in the verifier. This duplication was intentional: it tested whether serialization and MQTT transport preserved the result and record attribution, not whether two different properties were needed.

Table 6.4: E2 results.

Check	Result
Local / remote speed verdicts	40 / 40 identical
Reset-effect verdicts	39, alternating as designed
Reset attribution	service response and odometry in every verdict
Fibonacci action evidence	23 goals, 115 feedback records, 23 succeeded statuses
Total common records	1458

There was no infrastructure failure. Negative reset verdicts were expected stimuli. E2 satisfies the remaining observation part of C1 and shows for C2/C3 that one recorded stream retains its meaning across the local MQTT boundary.

6.6 | E3: One Nav2 Robot

E3 used an unmodified TurtleBot3 Nav2 application in Gazebo. Gazebo, Nav2, the mission process, monitor, broker, and verifier ran as separate processes on one workstation. The monitor observed `/cmd_vel`, rate-limited fields from `/odom`, and feedback/status from `/navigate_to_pose`. It checked a 0.2 m/s speed property and:

$$\text{goal status} = \text{succeeded} \wedge \text{duration} \leq 35 \text{ s.}$$

The mission deliberately produced three outcomes. Goal A used an open lower route. Before B, an obstacle closed that route, forcing the upper route. Before C, both openings were closed.

Table 6.5: E3 navigation results.

Goal	Nav2 result	Observed route	Property verdict
A	Succeeded in 24.34 s	Lower opening	Positive
B	Succeeded in 69.72 s	Upper opening	Negative: deadline
C	Aborted in 33.10 s	Stayed left	Negative: aborted

The online B verdict was emitted when the 35-second deadline passed while the action was still active. The C verdict followed the observed aborted status. These are expected negative application outcomes, not monitor failures. Mission and Gazebo logs independently confirmed

all three. The run contained 15,833 records: 2422 velocity, 12,720 action, and 691 rate-limited odometry records. All 12 speed transitions and all three navigation verdicts matched the recorded inputs and outcomes. E3 therefore supports C2 on an existing application whose navigation code was unchanged.

6.7 | E4: Two Nav2 Robots

E4 ran two namespaced TurtleBot3/Nav2 instances in Gazebo. One monitor collected six topics: each robot's `cmd_vel`, `amcl_pose`, and `odom`. Separate speed properties required `linear.x ≤ 0.3` m/s. A fleet property required the distance between the two localized map-frame poses to remain at least 1.0 m. Each new pose was paired with the latest pose of the other robot.

The robots crossed and returned, deliberately creating two violation periods. All four navigation goals succeeded. From 3463 records, the robot speed properties produced 12 and 20 correct transitions. The separation property produced exactly two negative and two recovery verdicts, all matched by replay and by the independent Gazebo position log. Minimum localized and Gazebo distances were 0.587 and 0.623 m respectively. The four final physical positions were 0.050–0.073 m from their goals.

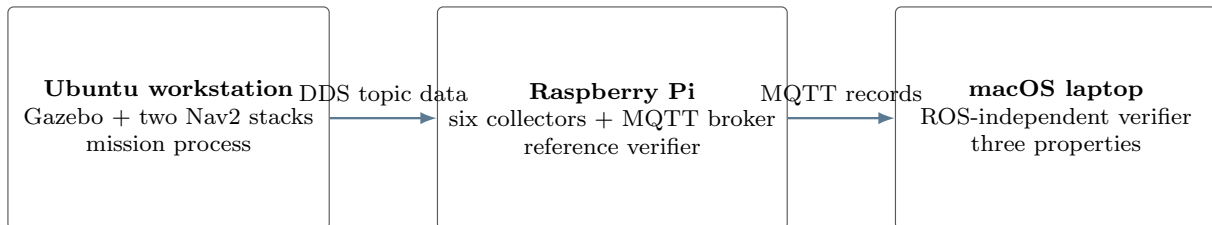
Table 6.6: E4 results.

Robot 1 / robot 2 speed verdicts	12 / 12; 20 / 20 correct
Separation transitions	4 / 4 correct
Online violation periods matched by Gazebo	2 / 2
Minimum localized / physical distance	0.587 / 0.623 m

E4 shows that source name keeps namespaces separate while a stateful converter can correlate observations from both robots.

6.8 | E5: Three Physical Machines

E5 retained the E4 mission and properties but changed placement, as shown in Figure 6.1. The Ubuntu workstation ran Gazebo, two Nav2 stacks, and the mission. A Raspberry Pi received the six topics over DDS, created common records, and hosted Mosquitto. A macOS laptop received MQTT records and ran the verifier without ROS 2. A second verifier on the Pi was a comparison reference only; it was not a required production component.



The Pi reference verifier receives the same MQTT records only to compare verdict streams.

Figure 6.1: E5 placement and the purpose of the Pi reference verifier.

All four goals succeeded. The Pi collected 3667 records, and both verifiers received all 3667. The Mac produced 12 and 18 correct speed transitions and four correct separation

transitions. Its three verdict streams were identical to the Pi reference, including contributing record identifiers. Both online violation periods matched Gazebo. Minimum localized and physical separations were 0.547 and 0.621 m, and all final positions were within 0.10 m of their goals.

Table 6.7: E5 correctness and delivery.

Check	Result
Mac robot 1 / robot 2 speed verdicts	12 / 12; 18 / 18 correct
Mac separation transitions	4 / 4 correct
Pi-reference / Mac verdict streams	all three identical
MQTT delivery to each verifier	3667 / 3667 records
Online violation periods matched by Gazebo	2 / 2

E5 supports C3: the collector and checker preserved the same result across the tested operating systems, processor architectures, DDS boundary, and MQTT boundary.

6.9 | Resource Use and Latency

Table 6.8 replaces a dispersed prose list with one summary. CPU values are percentages of one core. The measurements describe the reference runs only.

Table 6.8: Observed resource use and record-to-verdict latency.

Run/role	CPU	Memory	Mean or observed latency
E1 monitor	about 1%	about 73 MB	0.29 ms mean
E2 local	not isolated	not isolated	1.85 ms mean
MQTT path			
E4 monitor	below 5%	92 MB	a few milliseconds locally
E5 Pi	within 13%	114 MB	a few milliseconds over LAN
collection, broker, reference			
E5 Mac	about 1.4%	28 MB	a few milliseconds over LAN
verifier			

The results show modest cost for the evaluated sources and properties; they do not establish scalability beyond two simulated robots.

6.10 | Automation and Reused Work

To evaluate C4, Table 6.9 compares responsibilities, not speculative lines of code. Without the framework, each case study would need application-specific plumbing for these responsibilities. With the framework, only source/property/deployment choices and property-specific plug-ins remain.

The ten generated runtime YAML files and the unchanged E4-to-E5 property code are concrete outcomes of this separation. The framework automates routine implementation work while leaving domain knowledge visible.

Table 6.9: Recurring work avoided by the framework.

Responsibility	Without reusable infrastructure	With this framework
ROS 2 observation	Write topic, service-event, and hidden-action subscribers	Select source kind, name, type, and phases
Data interchange	Define serializers, source context, identities, and correlation	Reuse common record and verdict formats
Reduction/routing	Write field/rate/change logic and fan-out	Configure built-in transformers and graph links
Split placement	Implement publisher/subscriber transport and align endpoints	Declare one link; generate paired file/MQTT endpoints
Evidence	Design logs and verdict attribution	Reuse JSONL, session, record, and input identifiers
Property meaning	Implement application-specific checking	Still authored as converter/verdict plug-ins; not automated

6.11 | Accommodation of Existing Approaches

The four user-prepared mapping sketches were reduced to Figure 6.2 and Table 6.10. The purpose is not to claim equivalence or perform a detailed competitive comparison. It is to check that the reference architecture has compatible responsibilities and extension points.

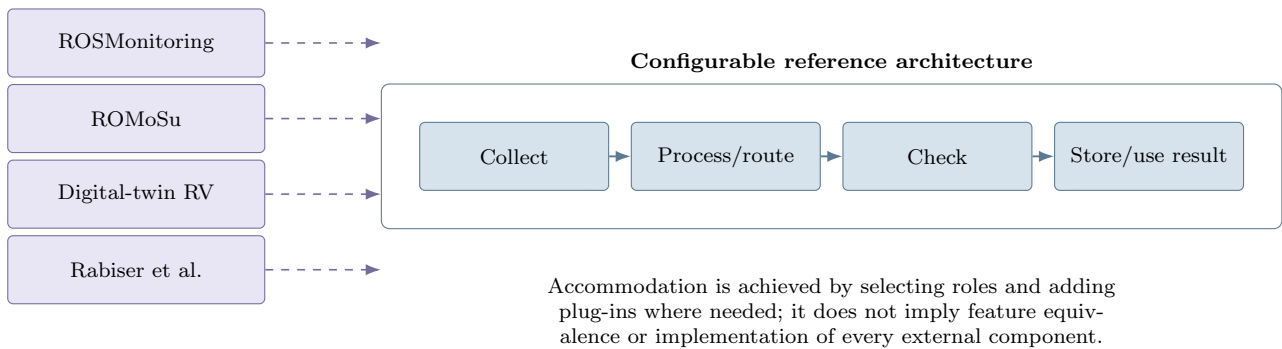


Figure 6.2: High-level accommodation of four reviewed approaches.

This mapping satisfies C5 at reference-architecture level. It does not turn those external systems into implemented prototype features.

Table 6.10: How principal capabilities can be hosted by the reference architecture.

Approach	Architecture configuration	Boundary
ROSMonitoring	ROS collector; converter to oracle events; external verdict-service adapter; optional feedback adapter for filtering	Feedback is architecture-only in the prototype
ROMoSu	Deployment configuration; ROS collectors and transformers; broker transport; validation as verdict service; storage/dashboard as exporters	Administration UI is an external configuration client
Digital-twin RV	MQTT source; stateful converter representing twin state; TeSSLa checker behind a verdict service; optional feedback	Twin and TeSSLa plug-ins are not implemented
Rabiser et al.	Specification/generation covers setup; collectors, transformers, dispatchers, and verdict services cover execution; exporters cover monitoring support	Domain model is broader than the selected prototype

6.12 | Answer to RQ4

Within the evaluated scope, the answer is positive. C1 is supported by E1–E3, which collect topics, service request/response events, and action feedback/status. C2 is supported by expected stimulus transitions, independent mission and Gazebo logs, replay, and verdict attribution. C3 is supported by integrated, split same-host, and three-machine placements. C4 is supported by ten generated runtime configurations and reuse of the same observation, transport, and evidence code, including unchanged E4 property logic in E5. C5 is supported by the high-level instantiations above.

The claim is limited to ROS 2 Kilted, the selected interfaces and plug-ins, one local network, up to two simulated robots, and runs lasting a few minutes.

Chapter 7

Discussion

This chapter interprets the results by research question, then discusses trade-offs, limitations, threats to validity, and future work.

7.1 | RQ1: Capabilities and Observation Gaps

RQ1: What monitoring capabilities and ROS 2 observation mechanisms exist, and which application-level observation gaps remain in the reviewed approaches?

ROS 2 can be observed through native application interfaces, application instrumentation, middleware or network probes, and execution tracing. These mechanisms answer different questions. Native interfaces retain ROS names and types; network probes avoid normal subscriptions; tracing reveals callbacks and scheduling; instrumentation can expose domain events that communication does not reveal.

The reviewed ROS-focused approaches cover topic observation most consistently. ROSMonitoring 2.0 adds services with partial ROS 2 support. In the selected literature, ROS 2 action feedback and status are not combined with typed topic and service-event collection in one configurable infrastructure. Other work provides generated checkers, digital-twin state, dashboards, network security, or lower-level tracing. The resulting gap is therefore not “monitoring in general”, but the reusable application-level path from heterogeneous ROS 2 observations to replaceable property checkers.

The prototype addresses that focused gap. E2 collected topic messages, service request/response events, and action feedback/status in one session; E3 used the same action mechanism with Nav2. A complete action trace remains incomplete because goal, cancellation, and result service events require additional service collectors.

7.2 | RQ2: Commonalities and Variabilities

RQ2: What are the commonalities and variabilities of monitoring solutions for ROS 2 applications?

The common operational tasks are collection, representation, processing, routing, checking, and use or storage of results. The principal variabilities are:

- observable element and observation level;
- passive and/or active behaviour;
- online and/or offline execution;
- integrated, split, centralised, or more distributed placement;

- property formalism and checker input format;
- transport, output, and evidence choices; and
- configured, generated, and plug-in-based implementation.

The feature model separates these dimensions. This matters because, for example, a checker formalism should not determine whether collection is local or remote, and an MQTT choice should not determine how a ROS message is observed. The model also makes the prototype boundary visible: filled features are evaluated, while optional features remain design points rather than implemented claims.

7.3 | RQ3: Reference Architecture

RQ3: How can a reference architecture incorporate the selected monitoring features while separating reusable infrastructure from property-specific checking?

The reference architecture uses a stable common data record between ROS-specific observation and property-specific checking. Collectors preserve source type, name, phase, message type, time, payload, session, and identity. Transformers and dispatchers perform checker-independent reduction and routing. A converter plug-in adapts records to one DSL-record shape; a verdict-service plug-in evaluates the corresponding property. Verdict exporters retain the result and contributing record identifiers.

This division is the central design result. Runtime type resolution, recursive message conversion, JSONL serialization, routing, transport, and evidence are implemented once. Application knowledge remains explicit in the source selection, converter, property checker, and parameters. The deployment generator then changes placement without changing those interfaces.

The plug-in story has a limited but precise scope. A plug-in is not an automatically generated checker. It is a Python class developed against one of two base interfaces and loaded by class path. Supporting another DSL requires a converter that creates its input and a verdict service that wraps its checker. The reference architecture can host such an adapter; the prototype provides small examples, not integrations with every formalism named in the feature model.

The architectural accommodation in Chapter 6 further checks the role boundaries. ROS-Monitoring’s external oracle, ROMoSu’s broker services, digital-twin state and TeSSLa checking, and the broader Rabiser et al. tasks all have places in the architecture. Components such as a dashboard, digital twin, TeSSLa engine, or feedback controller would still have to be implemented or connected.

7.4 | RQ4: Evaluation of the Selected Path

RQ4: To what extent do the proposed reference architecture and its implementation support the monitoring needs of representative ROS 2 applications?

The five experiments provide increasing evidence. E1 establishes the integrated topic path. E2 adds service-event and action observation and shows identical local and MQTT-transported speed verdicts. E3 monitors unmodified Nav2 and correctly distinguishes success, a missed deadline, and an aborted action. E4 keeps two namespaces separate while combining their localized poses. E5 moves collection and checking across Ubuntu, Raspberry Pi, and macOS without changing the E4 property logic.

Verdicts agreed with controlled expectations, replay, mission logs, or Gazebo positions in the reported runs. Record identifiers provide an evidence trail. The generated configurations and responsibility analysis also show that the infrastructure removes repeated observation,

serialization, transport, and evidence work. The answer is positive for the selected interfaces and deployments, not for the whole feature model or production-scale fleets.

7.5 | Design Trade-offs

The principal trade-offs are:

- **Early reduction versus complete evidence.** Field extraction, rate limiting, and change filtering reduce storage and network load, but may discard a short-lived value or a field needed by a later property. Applied transformer names are retained, but discarded observations cannot be reconstructed.
- **Integrated versus split placement.** Integrated checking avoids a transport boundary and has lower latency, but shares dependencies and resources with the ROS 2-facing monitor. Split checking isolates the checker and permits a ROS-independent host, but introduces broker, queue, delay, and disconnection concerns.
- **Uniform records versus source specificity.** One record interface simplifies processing and transport. Source type, phase, message type, and metadata must nevertheless be preserved so services and actions are not reduced to anonymous JSON.
- **Plug-in flexibility versus integration work.** Stable plug-in interfaces permit different DSLs and checkers. Users must still define, implement, and test the converter/checker contract.
- **Generation versus explicit decisions.** Generating YAML removes inconsistent wiring, but the deployment specification still requires knowledge of the ROS graph, property inputs, hosts, and network.

7.6 | Limitations

7.6.1 Observation and Checking

The implementation is passive and application-level. A topic collector sees only messages delivered to its subscription and therefore depends on DDS discovery and compatible QoS. Service events appear only when introspection is enabled. The action collector covers feedback and status, not the complete goal/cancel/result trace. Parameters, lifecycle transitions, DDS events, network packets, and operating-system traces are not implemented sources.

The property language is not generated or validated by the infrastructure. Stateful global properties are coded in converters and verdict services. Automatic property decomposition among monitors is outside scope, and checking remains logically centralised.

7.6.2 Ordering, Time, and Delivery

A record has monitor wall-clock time, an optional ROS timestamp, and a per-source sequence. The infrastructure does not establish a global order across publishers or synchronize host clocks. The E4/E5 separation converter pairs a new position with the latest received position of the other robot, so the result includes sampling and arrival-time differences.

MQTT QoS and record identifiers help with transport, but do not provide end-to-end exactly-once semantics. Bounded queues may drop data; reconnecting QoS 1 clients may observe duplicates. Persistent buffering, duplicate suppression, late-record policies, failover, access control, and encrypted deployment were not evaluated.

7.6.3 Configuration and Operation

The generator validates structure and aligns endpoints; it cannot decide whether a selected property or field is meaningful. Required ROS interface packages must be installed near collection. Runtime graphs are fixed for a run, so changes in discovered nodes or host placement require restart or explicit reconfiguration.

7.7 | Threats to Validity

7.7.1 Internal Validity

The deterministic stimuli, JSONL replay, mission logs, and Gazebo positions provide several checks of correctness. Some replay comparisons nevertheless use property logic closely related to the online implementation, so a shared error could affect both. Independent mission and Gazebo evidence reduce this threat for navigation outcomes and separation, but not for every field conversion.

Resource results are derived from short reference runs and may be affected by background workload, simulation timing, and measurement method. They are used only as indicative measurements. The feature model and architecture were developed by the thesis author; independent expert validation was not performed.

7.7.2 External Validity

The literature selection may have missed related or industrial work, and publications expose capabilities at different levels of detail. This directly limits RQ1 and RQ2 and is why absent capabilities are described as gaps in the *reviewed* work rather than universal absence.

Experiments use ROS 2 Kilted, Fast DDS, Mosquitto, one local network, at most two simulated TurtleBot3 robots, and runs lasting a few minutes. Other ROS 2 distributions, DDS vendors, networks, robot hardware, fleet sizes, and long-running operation may produce different discovery, delivery, latency, and resource behaviour. The selected cases sample only a small part of the feature model.

7.8 | Future Work

Future work is consolidated here:

- **Broader observation:** complete action traces; parameter and lifecycle events; DDS-level events; adapters for `ros2_tracing` and network observations.
- **Robust distribution:** persistent queues, duplicate suppression, replay after disconnection, clock synchronization, explicit late-record policies, multiple verifiers, health checks, failover, and security.
- **Easier DSL integration:** plug-in templates, a small conformance test harness, documented DSL-record contracts, and evaluated adapters for temporal-logic or state-machine checkers.
- **More transports and outputs:** supported plug-ins for sockets, HTTP/WebSocket, ROS 2 topics, databases, dashboards, and other brokers.
- **Guided configuration:** a Web interface that discovers the ROS graph, guides source, property, and placement selection, validates the resulting deployment specification, and exposes generated choices.

- **Stronger evaluation:** physical robots, larger fleets, long-duration runs, high-rate sensor data, network and broker failures, scalability measurements, and a user study of configuration effort.

Chapter 8

Conclusions

This thesis addressed the recurring infrastructure work between ROS 2 communication and property-specific checking. It reviewed the domain, developed a feature model and reference architecture, implemented the selected path, and evaluated five increasingly distributed experiments.

8.1 | Answers to the Research Questions

RQ1: What monitoring capabilities and ROS 2 observation mechanisms exist, and which application-level observation gaps remain in the reviewed approaches?

ROS 2 behaviour can be observed through native typed interfaces, instrumentation, network or middleware probes, and system tracing. Topic observation is common in the reviewed ROS-focused work, ROSMonitoring 2.0 adds services with partial ROS 2 support, and the reviewed approaches do not combine ROS 2 topic messages, service events, and action feedback/status in one configurable application-level infrastructure.

RQ2: What are the commonalities and variabilities of monitoring solutions for ROS 2 applications?

Solutions share collection, representation, processing, routing, checking, and result-use tasks. They vary in observable elements, observation level, monitoring mode, online/offline operation, placement, property formalism, transport, output, and whether artifacts are configured, generated, or extended through plug-ins. The feature model makes these choices explicit.

RQ3: How can a reference architecture incorporate the selected monitoring features while separating reusable infrastructure from property-specific checking?

A collector converts a ROS 2 observation to a common record. Reusable transformers and dispatchers reduce and route it. A converter plug-in adapts the record to a checker-specific DSL record, and a verdict-service plug-in evaluates the property. Common verdict and evidence formats retain attribution. The same roles can run together or be connected through a configured transport.

RQ4: To what extent do the proposed reference architecture and its implementation support the monitoring needs of representative ROS 2 applications?

Within the tested scope, the support is demonstrated. The experiments covered topics, service request/response events, action feedback/status, local and MQTT-transported checking, multi-source properties, two namespaces, and a three-computer deployment. Reported verdicts matched controlled expectations, replay, mission logs, or Gazebo evidence. Runtime YAML was generated from deployment specifications, and property logic was reused unchanged when E4 was moved to the E5 placement.

8.2 | Main Contribution

The main contribution is a clear architectural and implementation boundary between heterogeneous ROS 2 observations and checker-specific property logic. The common record preserves observation context, while configuration and plug-in interfaces allow sources, processing, checking, outputs, and placement to vary. Configuration generation reduces low-level routine boilerplate without hiding application and property decisions.

The result is intentionally a foundation rather than a universal verifier. Active feedback, complete action traces, lower-level observation, globally ordered distributed checking, and production-scale fault tolerance remain outside the prototype. The evaluation establishes feasibility and functional correctness for the reported configurations.

8.3 | Availability

The implementation, tests, documentation, and case-study material are available in the public repository: <https://github.com/Jin-Shikai/ROS2-Monitor-Infra>.

Bibliography

- [1] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. 5, 10
- [2] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. 5, 8, 12
- [3] Adrian Francalanza, Jorge A. Pérez, and César Sánchez. Runtime verification for decentralised and distributed systems. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 176–210. Springer, 2018. 5, 10, 15
- [4] Open Robotics. Topics vs services vs actions. ROS 2 Documentation. Accessed 2026-06-06. 8, 9
- [5] Open Robotics. Ros 2 topics. ROS 2 Documentation. Accessed 2026-06-06. 8
- [6] Open Robotics. Quality of service settings. ROS 2 Documentation. Accessed 2026-06-06. 8
- [7] Open Robotics. Configure service introspection. ROS 2 Documentation. Accessed 2026-06-06. 8
- [8] Geoffrey Biggs, Jacob Perron, and Shane Loretz. Actions. ROS 2 Design, 2019. Accessed 2026-06-06. 9
- [9] Rick Rabiser, Klaus Schmid, Holger Eichelberger, Michael Vierhauser, Sam Guinea, and Paul Grünbacher. A domain analysis of resource and requirements monitoring: Towards a comprehensive model of the software monitoring domain. *Information and Software Technology*, 111:86–109, 2019. 11
- [10] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, 1990. 11
- [11] David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35(6):615–636, 2010. 11
- [12] OASIS MQTT Technical Committee. MQTT version 5.0. OASIS Standard, 2019. Accessed 2026-06-25. 12
- [13] Angelo Ferrando, Rafael C. Cardoso, Michael Fisher, Davide Ancona, Luca Franceschini, and Viviana Mascardi. ROSMonitoring: A runtime verification framework for ROS. In *Towards Autonomous Robotic Systems*, volume 12228 of *Lecture Notes in Computer Science*, pages 387–399. Springer, 2020. 13

- [14] Maryam Ghaffari Saadat, Angelo Ferrando, Louise A. Dennis, and Michael Fisher. ROS-Monitoring 2.0: Extending ROS runtime verification to services and ordered topics. In *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems*, volume 411 of *Electronic Proceedings in Theoretical Computer Science*, pages 38–55, 2024. 14
- [15] Marco Stadler, Michael Vierhauser, and Jane Cleland-Huang. Towards flexible runtime monitoring support for ROS-based applications. In *Proceedings of the 4th International Workshop on Robotics Software Engineering*, pages 12–15. ACM, 2022. 14
- [16] Marco Stadler and Michael Vierhauser. ROMoS: Flexible runtime monitoring support for ROS-based applications. In *2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering*, pages 53–60, 2023. 14
- [17] Ivan Perez, Anastasia Mavridou, Tom Pressburger, Alexander Will, and Patrick J. Martin. Monitoring ROS2: From requirements to autonomous robots. In *Proceedings of the Fourth Workshop on Formal Methods for Autonomous Systems*, volume 371 of *Electronic Proceedings in Theoretical Computer Science*, pages 208–216, 2022. 14
- [18] Tomoya Yamaguchi, Bardh Hoxha, and Dejan Ničković. RTAMT: Runtime robustness monitors with application to CPS and robotics. *International Journal on Software Tools for Technology Transfer*, 26(1):79–99, 2024. 14
- [19] Christophe Bédard, Ingo Lütkebohle, and Michel R. Dagenais. `ros2.tracing`: Multipurpose low-overhead framework for real-time tracing of ROS 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022. 14
- [20] Sean Rivera, Antonio Ken Iannillo, Sofiane Lagraa, Clément Joly, and Radu State. ROS-FM: Fast monitoring for the robotic operating system (ROS). In *2020 25th International Conference on Engineering of Complex Computer Systems*, pages 15–24, 2020. 15
- [21] Elif Değirmenci, Yunus Sabri Kırca, İlker Özçelik, and Ahmet Yazıcı. ROSIDS23: Network intrusion detection dataset for robot operating system. *Data in Brief*, 51:109739, 2023. 15
- [22] Tommaso Puccetti, Simone Nardi, Cosimo Cinquilli, Tommaso Zoppi, and Andrea Ceccarelli. ROSPaCe: Intrusion detection dataset for a ROS2-based cyber-physical system and IoT networks. *Scientific Data*, 11(1):481, 2024. 15
- [23] Stefano Aldegheri, Nicola Bombieri, Samuele Germiniani, Federico Moschin, and Graziano Pravadelli. A containerized ROS-compliant verification environment for robotic systems. In *2021 Design, Automation and Test in Europe Conference and Exhibition*, pages 222–225, 2021. 15
- [24] Joakim Schack Betzer, Jalil Boudjadar, Mirgita Frasher, and Prasad Talasila. Digital twin enabled runtime verification for autonomous mobile robots under uncertainty, 2024. 15
- [25] Ricardo Caldas, Juan Antonio Piñera García, Matei Schiopu, Patrizio Pelliccione, Genáina Rodrigues, and Thorsten Berger. Runtime verification and field-based testing for ROS-based robotic systems. *IEEE Transactions on Software Engineering*, 50(10):2544–2567, 2024. 15

Appendix A

Schemas and Reproducibility

A.1 | Common Data Record

The serialized interchange object contains the following fields. Optional empty fields may be omitted.

Table A.1: Common data-record schema.

Field	Type	Meaning
<code>_type</code>	string	<code>data</code> , <code>session_start</code> , <code>session_end</code> , or <code>error</code> .
<code>session_id</code>	string	Identifier shared by one runtime session.
<code>record_id</code>	string	Stable observation identifier within the session.
<code>source_type</code>	string	<code>topic</code> , <code>service</code> , or <code>action</code> .
<code>source_name</code>	string	Fully qualified ROS 2 resource name.
<code>source_node</code>	string array	Discovered publisher or server node names, when available.
<code>msg_type</code>	string	ROS 2 interface type.
<code>phase</code>	string/null	Service <code>request/response</code> or action <code>feedback/status</code> ; absent for normal topics.
<code>timestamp</code>	number	Collector wall-clock receive time in seconds.
<code>ros_timestamp</code>	object/null	Optional <code>sec/nanosec</code> timestamp from the observation.
<code>data</code>	object	Recursively JSON-compatible payload.
<code>metadata</code>	object	Per-source sequence and processing context.

The following is a complete topic-record example:

```
{
  "_type": "data",
  "session_id": "20260727_101530_a1b2c3d4",
  "record_id":
    "20260727_101530_a1b2c3d4:topic:/cmd_vel:-:18",
  "source_type": "topic",
  "source_name": "/cmd_vel",
  "source_node": ["/controller_server"],
  "msg_type": "geometry_msgs/msg/Twist",
  "timestamp": 1785147330.274,
  "data": {
    "linear": {"x": 0.22, "y": 0.0, "z": 0.0},
```

```

    "angular": {"x": 0.0, "y": 0.0, "z": 0.14}
  },
  "metadata": {
    "seq": 18,
    "transformers_applied": ["FieldExtractor"]
  }
}

```

Listing A.1: Example common data record.

A.2 | Deployment Specification Example

This complete example places ROS 2 collection on `robot` and speed checking on `verifier`. The generator creates `robot.yaml` for `monitor_node` and `verifier.yaml` for `node_runner`, with matching MQTT endpoints.

```

{
  "hosts": [
    {
      "id": "robot",
      "runtimes": [
        {
          "id": "ros2_robot",
          "kind": "ros2",
          "sources": [
            {
              "id": "cmd",
              "source_kind": "topic",
              "name": "/cmd_vel",
              "interface": "geometry_msgs/msg/Twist"
            }
          ]
        }
      ],
    },
    {
      "id": "monitor",
      "kind": "monitor",
      "subscribe": ["cmd"]
    }
  ],
},
{
  "id": "verifier",
  "runtimes": [
    {
      "id": "speed_converter",
      "kind": "converter",
      "class_path": "custom.speed.CmdVelSpeedConverter",
      "input_from": ["cmd"],
      "output_to": "speed_dsl"
    },
    {
      "id": "speed_property",
      "kind": "verdict_service",
      "class_path": "custom.threshold.ThresholdVerdict",
      "input_from": ["speed_dsl"],
      "output_to": [
        {"kind": "stdout"},
        {"kind": "file",
          "path": "verdicts_{session_id}.jsonl"}
      ]
    }
  ],
}

```



```

        ],
        "property_id": "cmd_speed_limit",
        "field": "speed",
        "op": "<=",
        "threshold": 0.5
    }
]
}
],
"links": [
{
    "id": "robot_records",
    "from_host": "robot",
    "to_host": "verifier",
    "from_runtime": "monitor",
    "to_runtime": "speed_converter",
    "payload": "records",
    "transport": {
        "kind": "mqtt",
        "broker": "192.0.2.10",
        "port": 1883,
        "qos": 1,
        "topic_prefix": "monitor/"
    }
}
]
}

```

Listing A.2: Two-host deployment specification.

A.3 | Plug-in Contracts

A converter subclasses `DataConverter` and implements:

```
def convert(self, record: DataRecord) -> Any | None:
    ...
```

A verdict service subclasses `VerdictService` and implements:

```
def evaluate(self, dsl_record: Any) -> Verdict | None:
    ...
```

The converter must preserve the input record identifier in its DSL record when the resulting verdict needs attribution. The verdict service returns the common `Verdict` object; output destinations are configured separately.

A.4 | Repository

The implementation, tests, documentation, and case-study material are available at:

<https://github.com/Jin-Shikai/ROS2-Monitor-Infra>

The repository documentation describes the common record, runtime YAML, deployment generation, custom plug-ins, and reproducible execution commands.